UADER FCyT

IDTILAB
Laboratorio de Investigación y Desarrollo en Tecnologías Informáticas

# Trabajo Final Integrador - Museo de Redes Neuronales

## Licenciatura en Sistemas de Información

**Cátedra:** Inteligencia Artificial

**Integrantes:**

Mauro Mottet: Líder : responsable del código y repositorio
Agustín Scheffler: Responsable de arquitectura
Candela Rojas: Responsable de documentación
Florencia Colazo: Responsable de test

**Equipo docente:**

Dra. Daniela López De Luise
Alejandro Pérez

**Fecha:** 19/10/2026

## 1. Objetivo

Este informe presenta las corridas de CarGross.py con los dos datasets entregados y evalúa sus resultados. La definición del problema y la justificación de la elección de la red se desarrollan en las secciones 1.1 y 1.2 del Manual de Referencia. Allí, una anomalía se define de forma operativa como la creación de un clúster nuevo: un comportamiento que la red no logra asimilar a los prototipos aprendidos durante el régimen nominal.

El informe busca responder tres preguntas:

1. ¿La red descubre, sin etiquetas, estados operativos coherentes del servidor?
2. ¿Cómo influyen en el agrupamiento la vigilancia ρ y el orden de presentación de los patrones?
3. Si la red ya aprendió el régimen nominal, ¿reconoce como nuevo un régimen de estrés?

## 2. La red y su implementación

### 2.1 Algoritmo

La red tiene dos capas: la capa de entrada F1, con un nodo por variable, y la capa de salida F2, con un nodo por categoría (clúster). Cada categoría se representa con un prototipo: un patrón binario que resume a los patrones del clúster y que se guarda en los pesos descendentes (*top-down*, t), que van de F2 a F1. Los pesos ascendentes (*bottom-up*, b), que van de F1 a F2, determinan qué nodo compite primero.
La implementación sigue el algoritmo del Box 3 de Lippmann (1987). Para cada patrón binario X de N componentes:

1. Inicialización: cada nodo de salida j arranca libre (no comprometido con ninguna categoría), con pesos $t_{ij} = 1$ y $b_{ij} = 1/(1+N)$.
2. Puntaje de coincidencia: $\mu_j = \Sigma\ b_{ij} \cdot x_i$ para todos los nodos, comprometidos o libres; es la suma de los pesos ascendentes en las posiciones donde el patrón tiene un 1.
3. Ganador: el nodo de mayor $\mu_j$. En la red, esto ocurre por inhibición lateral: los nodos de F2 compiten y se inhiben entre sí hasta que queda uno solo activo.
4. Test de vigilancia: se calcula $\|T \cdot X\| / \|X\|$, es decir, la cantidad de unos del patrón que también están en el prototipo del ganador, dividida por la cantidad total de unos del patrón. Si ese valor supera ρ, el patrón se asigna al ganador.
5. Adaptación: el prototipo se actualiza con un AND lógico ($t_{ij} \leftarrow t_{ij} \cdot x_i$), que conserva un 1 solo donde el prototipo y el patrón tienen 1, y $b_{ij} \leftarrow t_{ij} \cdot x_i / (0.5 + \Sigma\ t_{ij} \cdot x_i)$.
6. Reset: si no se supera el test, el ganador se inhabilita para ese patrón y se vuelve al paso 3. Cuando gana un nodo libre, se crea una categoría nueva.

### 2.2 Verificación de la implementación

- Ejemplo de Lippmann (Fig. 11): con letras C, E y F de 5 × 5 píxeles, ρ = 0.9 produce tres ejemplares. Con una vigilancia menor, C y E se fusionan en un ejemplar idéntico a C. Una F con un píxel faltante se acepta y degrada el ejemplar F por el AND. Es el mismo comportamiento que describe el artículo.
- Equivalencia con el Box 3 literal: una implementación literal del Box 3, con M nodos inicializados y compitiendo desde el principio, produce las mismas asignaciones que CarGross.py en los dos datasets y en 100 conjuntos aleatorios, para distintos valores de ρ y de M.
- El conjunto completo de 16 pruebas automáticas se ejecuta con python3 -m unittest -v.

## 3. Datos

La fuente es la telemetría de un servidor: 2243 registros tomados cada 300 segundos (5 minutos) durante 8 días, con 5 métricas continuas: CPU, memoria, E/S de disco, red entrante y red saliente. preprocesamiento_art1.py binariza cada métrica con su percentil 90 en todo el período, es decir, el valor que solo supera el 10 % más alto de las mediciones. La variable vale 1 cuando la métrica está por encima de ese umbral:

| Variable binaria | Métrica cruda | Umbral (P90) | Significado cuando vale 1 |
|---|---|---|---|
| cpu_alta | cpu_util_percent | 53.51 | Uso de CPU alto |
| ram_alta | mem_util_percent | 91.15 | Uso de memoria alto |
| disco_saturado | disk_io_percent | 12.59 | E/S de disco alta |
| red_in_alta | net_in | 46.11 | Tráfico de red entrante alto |
| red_out_alta | net_out | 36.61 | Tráfico de red saliente alto |

Estos umbrales también están en metadatos_variables.csv.

- 1634 registros (72.85 %) no tienen ninguna alerta y se descartan, porque el vector nulo no puede procesarse (el test de vigilancia dividiría por cero).
- Quedan 609 registros con actividad: 463 con 1 o 2 alertas simultáneas y 146 con 3 o más.
- Dataset 1, régimen nominal: 100 de los 463 registros con 1 o 2 alertas, elegidos al azar y en orden temporal. Se usó una semilla fija (42), un número que hace que la elección al azar sea siempre la misma, para que el dataset se pueda regenerar idéntico.
- Dataset 2, régimen de estrés: los primeros 100 registros con 3 o más alertas.
- Los datasets no comparten registros y en los dos varían las 5 variables.

Combinaciones presentes en cada dataset (orden: cpu_alta, ram_alta, disco_saturado, red_in_alta, red_out_alta):

**Dataset 1: régimen nominal**

| Patrón | Registros | % |
|---|---|---|
| [0, 0, 1, 0, 0] | 23 | 23% |
| [1, 0, 0, 0, 0] | 21 | 21% |
| [0, 0, 0, 1, 1] | 21 | 21% |
| [1, 0, 1, 0, 0] | 18 | 18% |
| [0, 1, 0, 0, 0] | 12 | 12% |
| [1, 1, 0, 0, 0] | 3 | 3% |
| [0, 1, 1, 0, 0] | 1 | 1% |
| [0, 0, 1, 0, 1] | 1 | 1% |

**Dataset 2: régimen de estrés**

| Patrón | Registros | % |
|---|---|---|
| [0, 1, 0, 1, 1] | 47 | 47% |
| [1, 1, 1, 0, 0] | 17 | 17% |

| Patrón | Registros | % |
|---|---|---|
| [1, 0, 1, 1, 1] | 10 | 10% |
| [1, 1, 0, 1, 1] | 8 | 8% |
| [0, 0, 1, 1, 1] | 5 | 5% |
| [1, 0, 0, 1, 1] | 5 | 5% |
| [0, 1, 1, 1, 1] | 4 | 4% |
| [1, 1, 1, 1, 1] | 4 | 4% |

En ambos regímenes la red entrante y la saliente aparecen casi siempre juntas: en el nominal, la única excepción es un registro [0, 0, 1, 0, 1].

## 4. Metodología

Todas las corridas se hicieron con Python 3.12 y son deterministas: con los mismos datos y parámetros dan siempre el mismo resultado. Se reproducen con:

```
python3 CarGross.py -d dataset_telemetria_base.csv -p 0.5
python3 CarGross.py -d dataset_telemetria_estres.csv -p 0.5
python3 correr_experimentos.py      # genera resultados/*.csv
```

| Experimento | Qué se mide | Archivo |
|---|---|---|
| 1. Sensibilidad a la vigilancia | M (cantidad de categorías), tamaños y prototipos para ρ = 0.1, 0.3, 0.5, 0.7 y 0.9 | vigilancia.csv |
| 2. Evolución de M | Categorías acumuladas a medida que se presentan los patrones (ρ = 0.5) | evolucion_M.csv |
| 3. Efecto del orden | M y tamaños con 20 órdenes aleatorios (semillas 0 a 19), ρ = 0.3 y 0.5 | efecto_orden.csv |
| 4. Base y luego estrés | Pureza respecto del régimen de origen y patrones de estrés absorbidos por categorías nominales | base_luego_estres.csv |

Como no existen etiquetas reales de anomalía, la pureza se mide contra el régimen de origen de cada registro (nominal o estrés). Para cada clúster se cuenta cuántas instancias pertenecen a su régimen mayoritario; esos conteos se suman y se dividen por el total de registros. Una pureza de 1 significa que ningún clúster mezcla registros de los dos regímenes.

## 5. Resultados

### 5.1 Sensibilidad a la vigilancia

La tabla muestra la cantidad de categorías (M) y el tamaño de cada clúster, en el orden en que la red los creó:

| ρ | M nominal | Tamaños (nominal) | M estrés | Tamaños (estrés) |
|---|---|---|---|---|
| 0.1 | 4 | 43 / 24 / 12 / 21 | 2 | 21 / 79 |
| 0.3 | 4 | 43 / 24 / 12 / 21 | 2 | 21 / 79 |

| ρ | M nominal | Tamaños (nominal) | M estrés | Tamaños (estrés) |
|---|---|---|---|---|
| 0.5 | 8 | 23 / 21 / 18 / 12 / 3 / 1 / 1 / 21 | 4 | 19 / 12 / 57 / 12 |
| 0.7 | 8 | 23 / 21 / 18 / 12 / 3 / 1 / 1 / 21 | 5 | 17 / 59 / 15 / 7 / 2 |
| 0.9 | 8 | 23 / 21 / 18 / 12 / 3 / 1 / 1 / 21 | 7 | 17 / 8 / 6 / 49 / 11 / 5 / 4 |

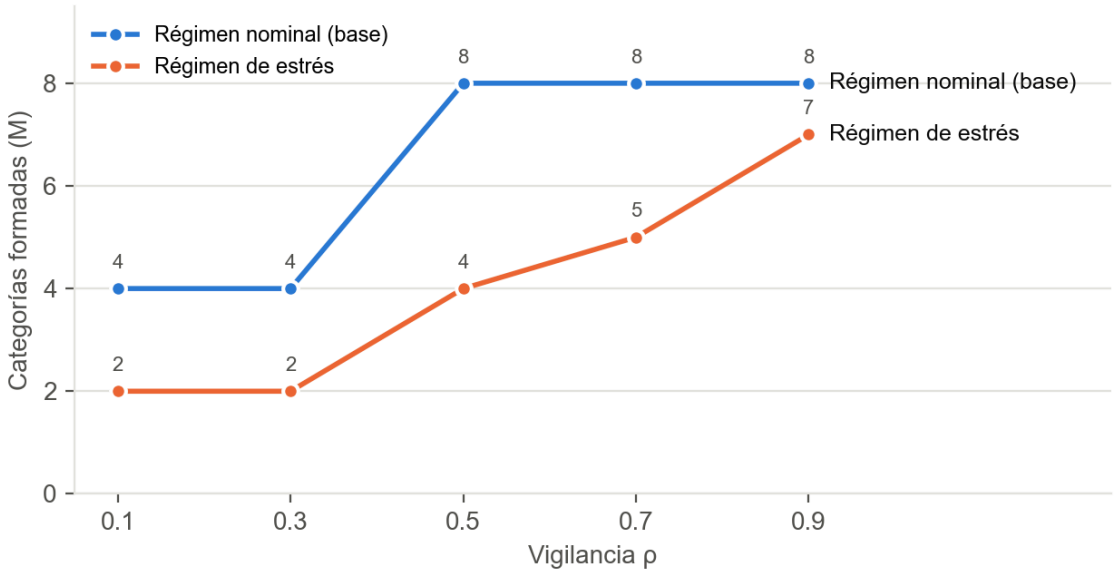



Figura 1. Categorías formadas (M) según la vigilancia ρ en cada dataset.
Prototipos aprendidos con ρ = 0.5 (salida del test demo del manual):

**Régimen nominal, ρ = 0.5**

| Nodo F2 | Prototipo top-down | Instancias |
|---|---|---|
| [00] | [0, 0, 1, 0, 0] | 23 |
| [01] | [1, 0, 0, 0, 0] | 21 |
| [02] | [1, 0, 1, 0, 0] | 18 |
| [03] | [0, 1, 0, 0, 0] | 12 |
| [04] | [1, 1, 0, 0, 0] | 3 |
| [05] | [0, 1, 1, 0, 0] | 1 |
| [06] | [0, 0, 1, 0, 1] | 1 |
| [07] | [0, 0, 0, 1, 1] | 21 |

**Régimen de estrés, ρ = 0.5**

| Nodo F2 | Prototipo top-down | Instancias |
|---|---|---|
| [00] | [1, 1, 1, 0, 0] | 19 |
| [01] | [0, 0, 0, 1, 1] | 12 |
| [02] | [0, 1, 0, 1, 1] | 57 |

| Nodo F2 | Prototipo top-down | Instancias |
|---|---|---|
| [03] | [1, 0, 1, 1, 1] | 12 |

**Régimen de estrés, ρ = 0.3**

| Nodo F2 | Prototipo top-down | Instancias |
|---|---|---|
| [00] | [1, 1, 1, 0, 0] | 21 |
| [01] | [0, 0, 0, 1, 1] | 79 |

### 5.2 Evolución de M

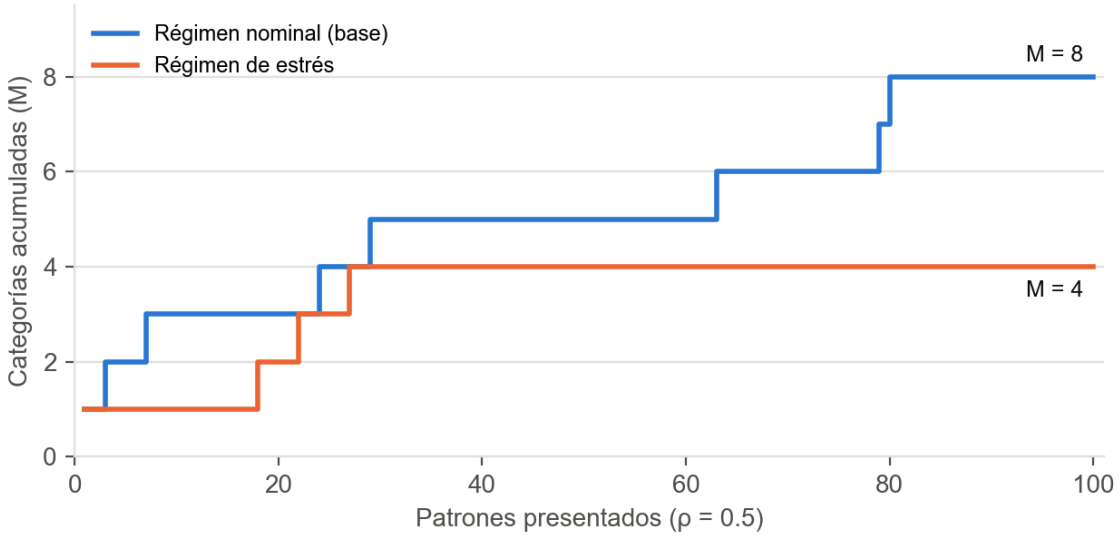



Figura 2. Categorías acumuladas a medida que se presentan los patrones (ρ = 0.5).
En el régimen de estrés las 4 categorías aparecen en los primeros 27 patrones y la red queda estable el resto de la secuencia. En el régimen nominal se crean categorías nuevas hasta el patrón 80, porque las combinaciones raras (entre 1 % y 3 % de los registros) aparecen tarde y cada una abre un nodo propio.

### 5.3 Efecto del orden de presentación

Se repitió la corrida con 20 órdenes aleatorios de presentación. La última columna indica cuántas particiones distintas se obtuvieron, es decir, cuántos repartos diferentes de los registros entre los clústeres:

| Dataset | ρ | M observado en 20 órdenes | Particiones distintas |
|---|---|---|---|
| Nominal (base) | 0.3 | M = 4 (20 veces) | 6 de 20 |
| Nominal (base) | 0.5 | M = 8 (20 veces) | 8 de 20 |
| Estrés | 0.3 | M = 2 (12 veces), M = 3 (8 veces) | 8 de 20 |
| Estrés | 0.5 | M = 4 (20 veces) | 20 de 20 |

### 5.4 Régimen nominal y luego estrés

Se presentaron a la red los 100 registros nominales seguidos de los 100 de estrés, como si la red monitoreara el servidor y, después de aprender la operación normal, llegara un período de carga.

| ρ | M total | Pureza | Patrones de estrés absorbidos por categorías nominales |
|---|---|---|---|
| 0.1 | 5 | 0.810 | 96 de 100 |
| 0.3 | 5 | 0.810 | 96 de 100 |
| 0.5 | 10 | 0.885 | 27 de 100 |
| 0.7 | 13 | 1.000 | 0 de 100 |
| 0.9 | 15 | 1.000 | 0 de 100 |

## 6. Evaluación de los resultados

### 6.1 Los estados descubiertos son coherentes

Con ρ = 0.5 los prototipos se interpretan directamente como estados del servidor. En el régimen nominal la red separa alertas aisladas de disco (23 %), de CPU (21 %) y de memoria (12 %), la combinación CPU + disco (18 %) y un estado de tráfico de red alto en ambos sentidos (21 %). En el régimen de estrés el estado dominante es memoria + red alta (57 %), seguido de CPU + memoria + disco (19 %): dos tipos de carga claramente distintos, uno ligado a la red y otro al procesamiento.

### 6.2 Efecto de la vigilancia

- Régimen nominal: M salta de 4 a 8 entre ρ = 0.3 y ρ = 0.5 y no cambia después. La explicación es aritmética: los registros tienen 1 o 2 alertas, así que la coincidencia con un prototipo distinto vale como máximo 1/2 = 0.5. Como el test es estricto (exige superar ρ, no alcanza con igualarlo), con ρ ≥ 0.5 cada combinación termina en su propia categoría; con ρ < 0.5 los registros de 2 alertas se fusionan con prototipos de 1 alerta.
- Régimen de estrés: M crece de forma gradual (2, 4, 5 y 7). Con ρ ≤ 0.3 la red resume el régimen en dos grandes estados: procesamiento (CPU + memoria + disco) y red. Con ρ = 0.9 casi cada combinación tiene su nodo.
- Esto confirma lo que planteaba la propuesta: una vigilancia baja produce pocas categorías amplias y una alta, muchas categorías específicas.

### 6.3 Efecto del AND: generalización y erosión

En el régimen de estrés con ρ = 0.5, el prototipo del nodo 01 es [0, 0, 0, 1, 1], una combinación que no existe en el dataset (todos los registros tienen 3 o más alertas). Es la intersección de los patrones que absorbió: la red generalizó, es decir, conservó la característica común (tráfico de red alto) y descartó lo demás. Este mismo mecanismo vuelve a la red sensible al ruido, porque un bit perdido nunca se recupera en el prototipo; a este efecto se lo llama erosión.

### 6.4 Dependencia del orden

Con ρ = 0.5 la cantidad de categorías no cambió en 20 órdenes aleatorios, pero la distribución de los registros entre ellas sí: en el régimen de estrés, los 20 órdenes produjeron 20 particiones distintas. Con ρ = 0.3, en el régimen de estrés M fue 2 en 12 órdenes y 3 en los otros 8. Es la limitación esperable del algoritmo del líder: el primer patrón de cada clúster condiciona su prototipo.

### 6.5 Detección del régimen nuevo

- Con ρ ≥ 0.7 ningún patrón de estrés es absorbido por una categoría aprendida durante el régimen nominal y la pureza es 1. La llegada del régimen de estrés se

manifiesta como la creación de 5 categorías nuevas, que es exactamente la definición operativa de anomalía (Manual de Referencia, sección 1.1).

- Con ρ = 0.5 se absorben 27 de 100 patrones de estrés. Por ejemplo, los 17 registros CPU + memoria + disco caen en la categoría nominal CPU + disco, porque coinciden en 2 de sus 3 alertas (2/3 = 0.67, que supera 0.5).
- Con ρ ≤ 0.3 se absorben 96 de 100: la red ya no distingue el estrés de la operación normal.

Conclusión práctica: para monitorear este servidor, ρ = 0.7 separa sin errores los dos regímenes con 13 categorías. ρ = 0.5 sirve para resumir cada régimen por separado, pero no alcanza como detector.

## 7. Conclusiones

CarGross.py implementa correctamente el algoritmo de Lippmann (1987). Con los datos del servidor, la red agrupó sin ayuda estados de funcionamiento que tienen sentido. Cuanto mayor es la vigilancia ρ, más detallado es el agrupamiento: con ρ ≥ 0.7, la red separó por completo los patrones de estrés de los del régimen nominal. Como los dos regímenes se definieron según la cantidad de alertas, esto muestra que el agrupamiento es coherente con ese criterio, pero no prueba que la red detecte fallas reales. Como señala Lippmann, la red depende del orden de los datos y es sensible al ruido.

## REFERENCIAS


- Lippmann, R. P. (1987). An Introduction to Computing with Neural Nets. IEEE ASSP Magazine, 4(2), 4–22. (Modelo de Carpenter/Grossberg, algoritmo del líder y Box 3.)
- Bezdek, J. C. (1981). Pattern Recognition with Fuzzy Objective Function Algorithms. Plenum Press. (Fuzzy C-Means, usado en la comparación de la FAQ.)